

GEW RHINO/RLT UV SYSTEM INSTALLATION MANUAL

GEW RHINO API - Read/Write



Contact Details

Address: GEW (EC) Limited,
Crompton Way, Crawley,
West Sussex, RH10 9QR, UK.

Telephone: +44 (0) 1737 824500 – main switchboard
+44 (0) 1737 824510 – service and spares

Email: service@gewuv.com
spares@gewuv.com
sales@gewuv.com

Website: www.gewuv.com

Distributors: www.gewuv.com/contact

Copyright

Duplication (copying, printing, microfilm or other forms) and the electronic distribution of this document is only allowed with explicit permission of GEW (EC) Limited. GEW (EC) Limited reserves the right to change technical data without prior announcement. The general business conditions and the regulations of the licence agreement do apply. All rights are reserved.

Contents

1	RHINO API Package Contents	5
1.1	Introduction	5
1.2	Postman Exports Folder	5
1.3	API Simulator Program	6
1.4	Demo Application.....	7
2	RHINO API	8
2.1	RHINO Web Interface	8
2.1.1	Introduction.....	8
2.1.2	Method	8
2.2	Data Protocol (API).....	8
2.2.1	Requests	8
2.2.2	Responses	9
2.2.3	POST Data	10
2.3	Endpoints	10
2.3.1	The System API.....	10
2.3.1.1	System Status Request /api/system/	10
2.3.2	The Network API	11
2.3.2.1	Network Status Request /api/network/.....	11
2.3.3	The Press API	12
2.3.3.1	Press Request/api/press/{x}/	12
2.3.4	The UV Monitoring API	13
2.3.4.1	UV Sensor Request /api/uv/{x}/	13
2.3.4.2	UV Status Request /api/lamps/{x}/uv/.....	14
2.3.4.3	UV Serial Number Request /api/lamps/{x}/uv/serials/	15
2.3.5	The Lamps API.....	16
2.3.5.1	Lamp Status Request /api/lamps/{x}/	16
2.3.5.2	Lamp Counters Request /api/lamps/{x}/counters/	18
2.3.5.3	Lamp Power Ramping Request /api/lamps/{x}/powerramping/	19
2.3.5.4	Lamp Alerts Request /api/lamps/{x}/alerts/	20
2.3.5.5	Lamp Stop Request /api/lamps/{x}/stop/.....	21
2.3.5.6	Lamp Start Request /api/lamps/{x}/start/	22
2.3.5.7	Lamp Reset Request /api/lamps/{x}/reset/	22
2.3.5.8	Lamp Select Request /api/lamps/{x}/select/.....	22
2.3.5.9	Lamp Deselect Request /api/lamps/{x}/deselect/	22
2.3.5.10	Lamp Force Cool Request /api/lamps/{x}/cool/	22
2.3.5.11	Lamp Select and Start Request /api/lamps/{x}/selectstart/	23
2.3.5.12	Lamp Select and Stop Request /api/lamps/{x}/selectstop/	23
2.3.5.13	Lamp Close Shutters Request /api/lamps/{x}/close/	23
2.3.5.14	Lamp Open Shutters Request /api/lamps/{x}/open/	23
2.4	Safety	24
2.5	Additional Resources	25

Document Issue Status

Revision	Date	Comments
1.0	08.11.18	First issue - JNO.
2.0	03.10.19	Document title "GEW RHINO API - Read / Write" was "GEW RHINO API". - JNO

1 RHINO API Package Contents

1.1 Introduction

This pack contains all the information, files and links you should require to integrate with the GEW RHINO webAPI. Details of the different components in this pack and how they can help you with your integration are given below. Section 2.1 of this document gives a brief overview of the nature of the API and how to access it. It also gives details of all the endpoints and the parameters that they return or require to be used correctly.

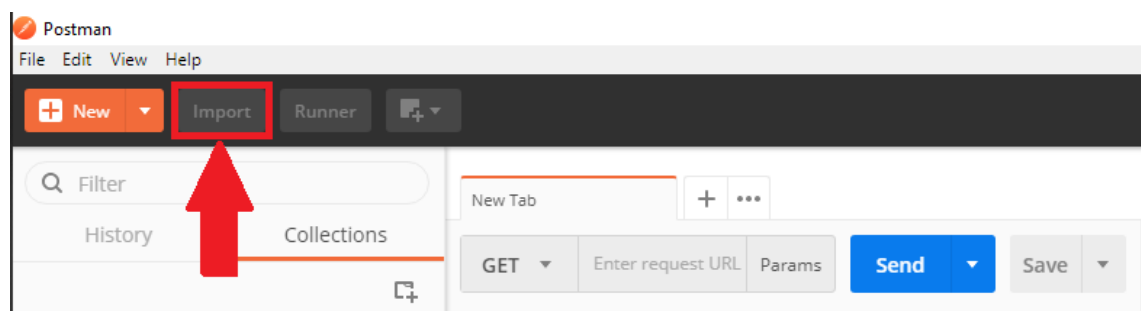
1.2 Postman Exports Folder

The files in this folder are exported from Postman. Postman is a desktop application for Windows, Mac and Linux that is used for building, testing and integrating with RESTful APIs. It can be downloaded for free from the following link:

<https://www.getpostman.com/apps>

Once installed, run the program. You do not need to make an account to continue.

From the main screen, you can import the files from this folder by clicking “Import” in the top left corner:

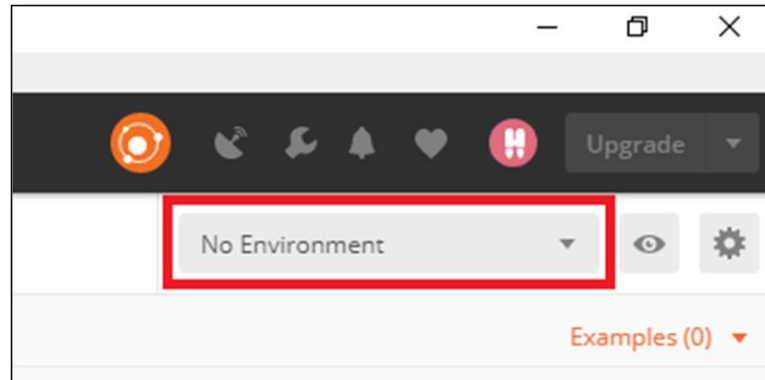


This will add a “Collection” and an “Environment” for you.

The “Collection” is the collection of endpoints that are available as part of the API and that postman can interact with for you.

Some parts of these endpoints have been replaced with variables, for instance notice the `{{address}}` as part of the endpoint URL. This is where the “Environment” comes in. The “Environment” defines these variables for us, allowing us to quickly and easily change certain parts of our requests. This allows for quickly changing between destinations, such as the local simulator or an actual system, or which lamp number to query.

Environments can be found in the top right corner of the Postman window:



By clicking the drop down, you should be able to select the “Local Simulator” environment that you imported. This is a preconfigured environment that allows you to interact the webAPI Simulator, detailed in section 1.3.

By clicking on the “eye” icon just next to this, you get the option to modify variables related to requests, such as the lamp number or system PIN.

You can manage your environments by clicking the “cog” icon. From here you can duplicate the Local Simulator environment so that you can create one that is specific to your system, allowing you to easily change between talking to your system and the simulator.



1.3 API Simulator Program

This is a program that can be used to do some basic integration testing before you have access to the actual system. It must be run as an administrator as it hosts a webserver to run the API on.

The default authorisation settings are:

Username: S11111

PIN: 366390

Any options are displayed on the right side of the window and must be set whilst the simulator is stopped. Attempting to start the simulator without admin privileges will result in the following message in the log window:

Unable to start Web API: Access is denied

The log window will give details about any POST requests that are received by the simulator and any parameters that are set as a result. GET requests can be checked by the response in either Postman or your own application.

All data responses are randomised from possible responses from an actual system. Any settings sent to the simulator are not remembered and should be verified by checking the console window of the simulator instead.

1.4 Demo Application

This is a C# WPF application developed in Visual Studio 2017 that you can look at to get an idea of how to integrate with the webAPI. This application is written to log the responses from a system with UV Monitors. Please bear in mind that this is for demonstration only; it was not designed with a production environment in mind, so may not be as robust as a press interfacing with the webAPI should be. Therefore, please take caution if reusing any code provided.

2 RHINO API

2.1 RHINO Web Interface

2.1.1 Introduction

The web interface provides a simple additional control option to the GEW RHINO system.

It is provided via a built-in webserver in the RHINO HMI, and enabled as required by GEW.

Please ensure you read and understand the Safety section of this document before proceeding.

2.1.2 Method

The HMI accepts HTTP connections on port 9910. You will need to ensure you are connected on the same network as the RHINO HMI, or connected directly via an Ethernet cable. To connect to the RHINO HMI you can visit in most modern web-browsers:

`http://GEW-S12345:9910/`

You will be prompted for a username (the system number) and password (provided by GEW).

2.2 Data Protocol (API)

An additional option is an HTTP 'RESTful' API for interfacing with the system.

A number of features can be provided through this interface which can be queried by the customer's applications, software, HMI, etc. The customer can write their own web-page displays to query data from one or more RHINO systems.

The HTTP 'RESTful' API is provided through port 9910. Requests must be authenticated using the same method as above via HTTP Basic Authentication.

GEW has a test 'API simulator' application that can be provided upon request for integration testing without a RHINO system present.

2.2.1 Requests

All requests against the API must be made to port 9910. Some API requests require a "GET" message (typically asking for information) and some require a "POST" message (typically performing an action).

If enabled, the API will require authorisation. A username (the system serial number, e.g. S12345) and password (a 6-digit PIN number provided by GEW) must be passed with each request. The type of authorisation is called "HTTP Basic Access Authentication" and is done via adding the "Authorization" entry to the HTTP header. This is detailed in the RFC1945 specification (<https://tools.ietf.org/html/rfc1945#section-10.2>).

To create the encrypted authentication string, encode the username:password using a base 64 encode algorithm. For example, S12345:000000 becomes UzEyMzQ1OjAwMDAwMA==.

In its rawest form, an HTTP packet is comprised of four ASCII lines, for example:

```
GET /api/system/ HTTP/1.1<CR><LF>
```

```
Host: GEW-S12345:9910<CR><LF>
```


Authorization: Basic UzEyMzQ10jAwMDAwMA==<CR><LF>
<CR><LF>

The server will simply reply with the response as ASCII text. For more information on basic HTTP packets, look here https://en.wikipedia.org/wiki/Hypertext_Transfer_Protocol#Example_session – other resources listed at the end of this document. Each line is split by a carriage return and linefeed, with an empty line (and therefore double CRLF) at the end.

2.2.2 Responses

Data can be returned in CSV or JSON (default) formats. The return type can be forced by specifying the “Content-Type” entry in the HTTP header. Available types are “text/json” or “text/csv”.

CSV data is returned with fixed lengths/padding. For numerical values data is padded with zeros, for string responses data is padded with spaces.

Boolean values are represented by true/false in JSON format, and 0/1 in CSV format.

All requests will receive an HTTP response; common codes are:

200: Request was received OK (note: if the request involved sending a message to a RHINO such as stop lamps, this status code means the message was transmitted OK but no guarantee it was received and acted upon by the power supply)

401: Authentication error (note: the password or username supplied was incorrect, or it is not permissible to perform this action at this time)

404: Invalid API endpoint (note: check the spelling or formatting of the API URL as the requested URL was not found)

500: Error (note: the request failed for an unspecified reason)

503: Service Unavailable (note: service is not yet ready, try again later)

All JSON responses will include some standard information:

```
1. {  
2.   "RequestURL": "/api/system/",  
3.   "CurrentDateTime": "02/07/2018 10:14:27",  
4.   "ResponseID": 8,  
5.   "Success": true,  
6.   "ExecutionTime": 0,  
7.   ...  
8. }
```

Parameter Name	Data Type	Range	Units	Description
RequestURL	String	N/A	N/A	The requested URL
CurrentDateTime	DateTime	N/A	YY/MM/DD hh:mm:ss	The date time of the response
ResponseID	Integer	≥0	Counts	An incremental counter for every response
Success	Boolean	N/A	N/A	Indicated if the request was successful
ExecutionTime	Integer	≥0	ms	The time taken to process the request

2.2.3 POST Data

Some POST requests require variables to be sent along with them to set the corresponding setting. These variables are sent in the body of the request in form of a query string:

body: ?Speed=60&Power=100

2.3 Endpoints

2.3.1 The System API

2.3.1.1 System Status Request /api/system/

An HTTP GET request to this endpoint will return a generic system status response. This can be used, for instance, to ensure you are connected to the correct RHINO system.

Parameters:

None

Example Response (JSON):

```

1. {
2.   ...
3.   "SystemSerial": "S12345",
4.   "SystemDescription": "E2C-67-4",
5.   "CurrentSoftware": "v1.10.01-d41d8cd98f00"
6. }
```

Example Response (CSV):

```

1. "SystemSerial","SystemDescription","CurrentDateTime","CurrentSoftware"
2. "S12345","E2C-45-
6","17/11/2016 11:50:13","v1.10.01-d41d8cd98f00"
```

Return Parameters:

Parameter Name	Data Type	Range	CSV Padded Length	Units
SystemSerial	String	N/A	6	N/A
SystemDescription	String	N/A	32	N/A
CurrentDateTime	DateTime	N/A	19	YY/MM/DD hh:mm:ss
CurrentSoftware	String	N/A	21	N/A

2.3.2 The Network API

2.3.2.1 Network Status Request /api/network/

An HTTP GET request to this endpoint will return a generic network status response. This can be used, for instance, to check the MAC or IP addresses of the attached HMI.

Parameters:

None

Example Response (JSON):

```

1. {
2.   ...
3.   "NetworkIsAvailable": true,
4.   "RemoteConnection":
5.     {
6.       "LastConnectionTime": "2018-07-02 10:52:46",
7.       "LastServerName": "http://cloud1.gewuv.com",
8.       "LastPublicIP": "1.2.3.4",
9.       "IsConnectionAlive": true,
10.      "HasValidRegistration": true
11.    },
12.   "IsUpdateAvailable": false,
13.   "CurrentSoftware": "v1.10.01-d41d8cd98f00"
14.   "NetworkInterfaces":
15.     [{
16.       "Name": "Local Area Connection 13",
17.       "Hostname": "GEW-S12345",
18.       "Status": "Up",
19.       "Address":
20.         [
21.           {
22.             "IPv4": "192.0.1.120",
23.             "NetworkMask": "255.255.0.0"
24.           }
25.         ],
26.       "IsDHCPEnabled": true,
27.       "PhysicalAddress": "00:04:5F:95:90:17",
28.       "DHCPServers": ["192.0.0.11"],
29.       "Gateway": ["192.0.0.254"],

```

```

28.         "NameServers": ["192.0.0.11", "192.0.0.7", "192.0.0.9"],
29.         "Speed": "100Mbps"
30.     }
31. }

```

Example Response (CSV):

Not Available

Response Parameters:

Parameter Name	Data Type	Range	CSV Padded Length	Units
NetworkIsAvailable	Boolean	N/A	N/A	N/A
RemoteConnection	Array	N/A	N/A	N/A
LastConnectionTime	DateTime	N/A	N/A	YY/MM/DD hh:mm:ss
LastServerName	String	N/A	N/A	N/A
LastPublicIP	String	N/A	N/A	N/A
IsConnectionAlive	Boolean	N/A	N/A	N/A
HasValidRegistration	Boolean	N/A	N/A	N/A
IsUpdateAvailable	Boolean	N/A	N/A	N/A
CurrentSoftware	String	N/A	N/A	N/A
NetworkInterfaces	Array	N/A	N/A	N/A
Name	String	N/A	N/A	N/A
Hostname	String	N/A	N/A	N/A
Status	String	Up/Down	N/A	N/A
Address	Array	N/A	N/A	N/A
IPv4	String	N/A	N/A	N/A
NetworkMask	String	N/A	N/A	N/A
IsDHCPEnabled	Boolean	N/A	N/A	N/A
PhysicalAddress	String	N/A	N/A	N/A
DHCPServers	Array of Strings	N/A	N/A	N/A
Gateway	Array of Strings	N/A	N/A	N/A
NameServers	Array of Strings	N/A	N/A	N/A
Speed	String	N/A	N/A	N/A

2.3.3 The Press API

2.3.3.1 Press Request/api/press/{x}/

An HTTP POST request to this endpoint allows the user to set the impression state and press speed. Note that if this option is enabled, the hardware control of these signals will be disabled.

Parameters:

- {x}:** The lamp number, between 1 and 16. Additionally, 0 will set data for all lamps.
- impression:** The impression state to set for the lamp. Either true or false
- speed:** The speed to set for the lamp, in the range 0 – 100 as a percentage of the maximum speed

2.3.4 The UV Monitoring API

2.3.4.1 UV Sensor Request /api/uv/{x}/

An HTTP GET request to this endpoint will return a custom sub-set of data about UV monitors on the system.

Parameters:

- {x}:** The lamp number, between 1 and 16. Additionally, 0 will return data for all lamps.

Example Response (JSON):

```

1. {
2.   ...
3.   "Lamp1":
4.     {
5.       "NodeAddress": 1,
6.       "Sensor1"
7.         {
8.           "Address": 17,
9.           "UnitId": "P01-030000001234",
10.          "FirmwareVersion": 4,
11.          "PowerOnHours": 24,
12.          "TemperatureMin": 18,
13.          "TemperatureAverage": 23,
14.          "TemperatureMax": 29
15.        },
16.       "Sensor2": {
17.         "Address": 18,
18.         ...
19.       }...
20.     }
21.   "Lamp2":
22.     {
23.       "NodeAddress": 2,
24.       ...
25.     }
26.   ...
27. }
```

Example Response (CSV):

Not available

Response Parameters:

Parameter Name	Data Type	Range	CSV Padded Length	Units
Lamp{x}	Array	x: [1-16]	N/A	N/A
NodeAddress	Integer	[1-16]	N/A	N/A
Sensor{x}	Array	x: [1-16]	N/A	N/A
Address	Integer	[16-31]	N/A	N/A
UnitId	String	N/A	N/A	N/A
FirmwareVersion	Integer	≥1	N/A	N/A
PowerOnHours	Integer	≥0	N/A	Hours
TemperatureMin	Integer		N/A	DegC
TermperatureAverage	Integer		N/A	DegC
TemperatureMax	Integer		N/A	DegC

2.3.4.2 UV Status Request /api/lamps/{x}/uv/

An HTTP GET request to this endpoint will return the live data for all attached sensors on that lamp and basic information on the lamp's running state.

Parameters:

{x}: The lamp number, between 1 and 16. Additionally, 0 will return data for all lamps.

Example Response (JSON):

```

1. {
2.   ...
3.   "Lamp1":
4.     {
5.       "NodeAddress": 1,
6.       "UnitInternalState": "Printing",
7.       "LampPowerPercent": 45,
8.       "LampPower": 4000,
9.       "PressImpression": true,
10.      "PressSpeed": 22,
11.      "IsSelected": true,
12.      "AlertCount": 0,
13.      "Sensor1Raw": 105,
14.      "Sensor2Raw": 55,
15.      "Sensor3Raw": 44,
16.      "Sensor4Raw": 5,
17.      "Sensor5Raw": 70,
18.      "Sensor6Raw": 0,
19.      "Sensor7Raw": 0,
20.      "Sensor8Raw": 0
21.    },
22.   "Lamp2":
23.     {
24.       "NodeAddress": 2,
```

```

25.         ...
26.     },
27.     ...
28. }

```

Example Response (CSV):

```

1. "NodeAddress","UnitInternalState","LampPowerPercent","LampPower","PressImp
   pression","PressSpeed","IsSelected","AlertCount","Sensor1Raw","Sensor2Raw",
   "Sensor3Raw","Sensor4Raw","Sensor5Raw","Sensor6Raw","Sensor7Raw","Sensor8R
   aw"
2. " 1","      Printing","016","00799","1","000","1","001","65535","65535","655
   35","65535","65535","65535","65535","65535"
3. ...
4. "16","      Offline","000","00000","0","000","0","000","00000","00000","000
   00","00000","00000","00000","00000","00000"

```

Response Parameters:

Parameter Name	Data Type	Range	CSV Padded Length	Units
Lamp{x}	Array	{x}: [1-16]	N/A	N/A
NodeAddress	Integer	[1-16]	2	N/A
UnitInternalState	Enum	“Offline”, “Boot”, “Init”, “Idle”, “Purge”, “BurnInDelay”, “Ignition”, “BurnIn”, “Ready”, “Printing”, “Cooling”	12	N/A
LampPowerPercent	Integer	[0-100]	3	%
LampPower	Integer	[0-27000]	5	Watts
PressImpression	Boolean	N/A	1	N/A
PressSpeed	Integer	[0-100]	3	%
IsSelected	Boolean	N/A	1	N/A
AlertCount	Integer	≥0	3	N/A
Sensor{x}Raw	Integer	{x}: [1-8] [0-232]	5	100x mW/cm2

2.3.4.3 UV Serial Number Request /api/lamps/{x}/uv/serials/

An HTTP GET request to this endpoint will return a list of the serial numbers for all attached peripherals on that lamp.

Parameters:

{x}: The lamp number, between 1 and 16. Additionally, 0 will return data for all lamps.

Example Response (JSON):

```

1. {
2.   ...
3.   "Lamp1":
4.     {
5.       "Sensor1Serial": "P01-010000000086",
6.       "Sensor2Serial": "P01-010000000087",
7.       "Sensor3Serial": "P01-010000000088",
8.       "Sensor4Serial": "P01-010000000089",
9.       "Sensor5Serial": "P01-010000000090",
10.      "Sensor6Serial": "",
11.      "Sensor7Serial": "",
12.      "Sensor8Serial": "",
13.      "NodeAddress": 1
14.    },
15.   "Lamp2":
16.     {
17.       "Sensor1Serial": "P01-010000000099",
18.       "Sensor2Serial": "P01-010000000100",
19.       ...
20.     },
21.   ...
22. }
```

Example Response (CSV):

```

1. "NodeAddress","Sensor1Serial","Sensor2Serial","Sensor3Serial"...
2. "01","P01-010000000086","", "P01-010000000088"...
3. ...
4. "16","P01-010000000121","P01-010000000122","P01-01000000013"..."
```

Response Parameters:

Parameter Name	Data Type	Range	CSV Padded Length	Units
Lamp{x}	Array	{x}: [1-16]	N/A	N/A
NodeAddress	Integer	[1-16]	2	N/A
Sensor{x}Serial	String	{x}: [1-16]	16	N/A

2.3.5 The Lamps API

2.3.5.1 Lamp Status Request /api/lamps/{x}/

An HTTP GET request to this endpoint will return a custom sub-set of data about ALL lamps on the system. Typically this request is the main API call used to check status of the system every x seconds, and then further API calls are made if

required (such as getting a list of alerts if this response returns “AlertCount” greater than 0).

Parameters:

{x}: The lamp number, between 1 and 16. Additionally, 0 will return data for all lamps.

Example Response (JSON):

```

1. {
2.   ...
3.   "Lamp1":
4.     {
5.       "NodeAddress": 1,
6.       "UnitInternalState": "Printing",
7.       "LampPowerPercent": 40,
8.       "LampPower": 5500,
9.       "PressImpression": true,
10.      "PressSpeed": 15,
11.      "IsSelected": true,
12.      "AlertCount": 0
13.    },
14.    "Lamp2":
15.      {
16.        "NodeAddress": 2,
17.        ...
18.      },
19.    ...
20.  }
```

Example Response (CSV):

```

1. "NodeAddress","UnitInternalState","LampPowerPercent","LampPower","PressImpression","PressSpeed","IsSelected","AlertCount"
2. "01","Printing","040","05500","1","015","1","000","16","Printing","040","05500","1","015","1","000"
```

Response Parameters:

Parameter Name	Data Type	Range	CSV Padded Length	Units
Lamp{x}	Array	{x}: [1-16]	N/A	N/A
NodeAddress	Integer	[1-16]	2	N/A
UnitInternalState	Enum	“Offline”, “Boot”, “Init”, “Idle”, “Purge”, “BurnInDelay”, “Ignition”, “BurnIn”, “Ready”, “Printing”, “Cooling”	12	N/A
LampPowerPercent	Integer	[0-100]	3	%
LampPower	Integer	[0-27000]	5	Watts
PressImpression	Boolean	N/A	1	N/A
PressSpeed	Integer	[0-100]	3	%
IsSelected	Boolean	N/A	1	N/A
AlertCount	Integer	≥0	3	N/A

2.3.5.2 Lamp Counters Request `/api/lamps/{x}/counters/`

- An HTTP GET request to this endpoint will return the lamp hour counters for this lamp.

Parameters:

{x}: The lamp number, between 1 and 16. Additionally, 0 will return data for all lamps.

Example Response (JSON):

```

1. {
2.   ...
3.   "Lamp1":
4.     {
5.       "NodeAddress": 1,
6.       "LampReadyHours": 120.9,
7.       "LampPrintingHours": 303.0,
8.       "LampTotalHours": 423.9,
9.       "LampIgnitionCount": 16
10.    },
11.   "Lamp2":
12.     {

```

```

13.         "NodeAddress": 2,
14.         ...
15.     },
16.     ...
17. }

```

Response (CSV):

```

1. "NodeAddress", "LampReadyHours", "LampPrintingHours", "LampTotalHours", "LampIgnitionCount"
2. " 1", "00121", "00303", "00424", "00016"
3. ...
4. "16", "00719", "01153", "01872", "00069"

```

Response Parameters:

Parameter Name	Data Type	Range	CSV Length	Padded	Units
Lamp{x}	Array	{x}: [1-16]	N/A		N/A
NodeAddress	Integer	[1-16]	2		N/A
LampReadyHours	JSON: 1dp CSV: Integer	≥0	5		Hours
LampPrintingHours	JSON: 1dp CSV: Integer	≥0	5		Hours
LampTotalHours	JSON: 1dp CSV: Integer	≥0	5		Hours
LampIgnitionCount	Integer	≥0	5		N/A

2.3.5.3 Lamp Power Ramping Request /api/lamps/{x}/powerramping/

- An HTTP GET request to this endpoint will return the Power vs Speed relationship for the lamp.
- There is also the option to submit an HTTP POST request which will allow the Power vs Speed relationship to be adjusted.

Parameters (GET):

- {x}: The lamp number, between 1 and 16. Additionally, 0 will return data for all lamps.

Parameters (POST):

{x}: The lamp number, between 1 and 16. Additionally, 0 will set the power speed for all lamps.

Speed: Specified in the request body, the new speed value desired from 0-100%.

Power: Specified in the request body, the new power value desired from 0-100%.

Response (JSON):

```

1. {
2.     ...

```

```

3.   "Lamp1":
4.   {
5.       "NodeAddress": 1,
6.       "Speed": 100,
7.       "Power": 20
8.   },
9.   "Lamp2":
10.  {
11.      "NodeAddress": 2,
12.      "Speed": 50,
13.      "Power": 80
14.  },
15.  ...
16. }

```

Response (CSV):

```

1. "NodeAddress","Speed","Power"
2. " 1","100","020"
3. ...
4. "16","100","100"

```

Response Parameters:

Parameter Name	Data Type	Range	CSV Padded Length	Units
Lamp{x}	Array	{x}: [1-16]	N/A	N/A
NodeAddress	Integer	[1-16]	2	N/A
Speed	Integer	[0-100]	3	%
Power	Integer	[0-100]	3	%

2.3.5.4 Lamp Alerts Request /api/lamps/{x}/alerts/

An HTTP GET request to this endpoint will return a list of all alerts (faults and warnings) present on the lamp(s). This should be called when “AlertCount” is greater than 0 to identify the types of fault(s) present on a lamp.

For a list of possible alerts, please refer to the RHINO System Manual.

Parameters:

{x}: The lamp number, between 1 and 16. Additionally, 0 will return data for all lamps.

Example Response (JSON):

```

1. {
2.   ...
3.   "Lamp1":
4.   {
5.       "NodeAddress": 1,
6.       "Alerts":
7.       [{
8.           "AlertCode": "eWarningCodeUvMonitorComms",

```

```

9.         "AlertTime": "7/3/2018 9:58:04 AM",
10.        "AlertNumber": 651,
11.        "IsFault": false,
12.        "IsWarning": true
13.    }],
14.    "AlertCount": 1
15. },
16. "Lamp2":
17. {
18.     "NodeAddress": 2,
19.     "AlertCount": 0
20. }
21. ...
22. }

```

Example Response (CSV):

```

1. "NodeAddress","AlertDescription","AlertCode","AlertTime","AlertNumber","Is
   Fault","IsWarning"
2. " 1",""                                ",""      eWarningCodeUvMonitorComms"
   ,"7/3/2018 9:58:04 AM","651","0","1"
3. ...
4. " 4",""                                ",""      eWarningCodeUvMonitorComms"
   ,"7/3/2018 9:58:05 AM","651","0","1"

```

Response Parameters:

Parameter Name	Data Type	Range	CSV Padded Length	Units
Lamp{x}	Array	{x}: [1-16]	N/A	N/A
AlertDescription	String	N/A	32	N/A
AlertCode	String	[0-100]	32	N/A
AlertTime	DateTime	N/A	3	YY/MM/DD hh:mm:ss AM/PM
AlertNumber	Integer	[0-999]	3	N/A
IsFault	Boolean	N/A	1	N/A
IsWarning	Boolean	N/A	1	N/A

2.3.5.5 Lamp Stop Request /api/lamps/{x}/stop/

An HTTP POST request to this endpoint will send a command to the RHINOs to stop one or more lamps.

Parameters:

{x}: The lamp number, between 1 and 16. Additionally, 0 will stop all lamps.

Response:

None

2.3.5.6 Lamp Start Request /api/lamps/{x}/start/

An HTTP POST request to this endpoint will send a command to the RHINOs to start one or more lamps. This API command must be additionally enabled by GEW due to the risk associated with remotely starting a system.

Parameters:

{x}: The lamp number, between 1 and 16. Additionally, 0 will start all lamps.

Response:

None

2.3.5.7 Lamp Reset Request /api/lamps/{x}/reset/

An HTTP POST request to this endpoint will send a command to the RHINOs to reset faults on one or more lamps. If trying to clear an Emergency Stop fault, it must be sent to all lamps (0) or it will fail to reset.

Parameters:

{x}: The lamp number, between 1 and 16. Additionally, 0 will reset all lamps.

Response:

None

2.3.5.8 Lamp Select Request /api/lamps/{x}/select/

An HTTP POST request to this endpoint will send a command to the RHINOs to select one or more lamps.

Parameters:

{x}: The lamp number, between 1 and 16. Additionally, 0 will select all lamps.

Response:

None

2.3.5.9 Lamp Deselect Request /api/lamps/{x}/deselect/

An HTTP POST request to this endpoint will send a command to the RHINOs to deselect one or more lamps.

Parameters:

{x}: The lamp number, between 1 and 16. Additionally, 0 will deselect all lamps.

Response:

None

2.3.5.10 Lamp Force Cool Request /api/lamps/{x}/cool/

An HTTP POST request to this endpoint will send a command to the RHINOs to force a cooldown cycle on one or more lamps. Lamps which are currently on/running will ignore this request.

Parameters:

{X}: The lamp number, between 1 and 16. Additionally, 0 will cool all lamps.

Response:

None

2.3.5.11 Lamp Select and Start Request /api/lamps/{x}/selectstart/

An HTTP POST request to this endpoint will send a command to the RHINOs to select themselves and then start straight away. Lamps which are currently on/running will ignore this request.

Parameters:

{x}: The lamp number, between 1 and 16. Additionally, 0 will select and start all lamps.

Response:

None

2.3.5.12 Lamp Select and Stop Request /api/lamps/{x}/selectstop/

An HTTP POST request to this endpoint will send a command to the RHINOs to select themselves but also issue a stop command. Lamps which are currently off/stopped will ignore this request.

Parameters:

{x}: The lamp number, between 1 and 16. Additionally, 0 will select and stop all lamps.

Response:

None

2.3.5.13 Lamp Close Shutters Request /api/lamps/{x}/close/

An HTTP POST request to this endpoint will send a command to the RHINOs to close the shutters on one or more lamps. Lamps which are currently on/running will ignore this request.

Parameters:

{x}: The lamp number, between 1 and 16. Additionally, 0 will close shutters on all lamps.

Response:

None

2.3.5.14 Lamp Open Shutters Request /api/lamps/{x}/open/

An HTTP POST request to this endpoint will send a command to the RHINOs to open the shutters on one or more lamps. Lamps which are currently on/running will ignore this request.

Parameters:

{x}: The lamp number, between 1 and 16. Additionally, 0 will open shutters on all lamps.

Response:

None

2.4 Safety

Using a remote API can present a safety issue due to allowing a remote user to interact with the system; potentially starting a lamp without being physically present in front of a machine.

Ethernet/API based control has many of the same problems that any remote interface would present (e.g. Profibus, RS-232), but also the additional concerns caused by being connected to an Ethernet network and therefore possibly accessible from anywhere, even outside the building.

There are several steps taken to prevent the use of a remote API such as this becoming a safety issue in practice:

- Local interlocks and fault circuits will always prevent the system from operating if not ready. The press emergency stop, safety stop and lamphead safety switch interlocks are hardware based and cannot be overridden by the software or API interface.
- The remote control functionality of the API is only enabled at the user's request. By default the API is 'read only' and can only be used to monitor the system rather than control it.
- The HMI and API work in tandem and reflect the true status of the lamp, regardless of who issued the most recent command. If the API request starts the lamps remotely, but the operator then stops the lamp, the lamps will still stop. Before working on the machine, the operator should switch off the RHINO system, or disable one of the safety interlocks to prevent a further remote start.
- The API has basic password protection through HTTP Authentication which can be used to prevent simple interference by an unauthorised user on the network. However, this is not considered a complete authentication solution and should only form part of the protection.
- When using the API with remote control functionality, the customer must undertake steps to prevent misuse, including:
 - Firewall and network routing rules to prevent any unauthorised users on the network from accessing the API.
 - Never exposing the API to the public internet by direct internet connection or port forwarding. The HMI must always be placed behind a suitable firewall/router device.
 - Using the HTTP Authentication features of the HMI to prevent simple interference by an unauthorised user on the network.
- Always ensuring the operator interacting with the API has direct line-of-sight to the machine.

- Never automating use of the API in a way that can create danger, e.g. programming timers to start and stop lamps at specific times.
- Verifying the API is interacting with the correct UV System before transmitting a command. This can be read back via the /system/ API command and prevents situations where a site has multiple systems and assigned IP addresses may for instance change over time.

2.5 Additional Resources

Some additional links that might help with implementation:

Introduction to RESTful APIs:

https://en.wikipedia.org/wiki/Representational_state_transfer

HTTP Basic Authentication: <https://tools.ietf.org/html/rfc1945#section-10.2>

Consuming RESTful API in .NET: <http://blog.anthonybaker.me/2013/05/how-to-consume-json-rest-api-in-net.html>

Consuming RESTful API in Java: <https://spring.io/guides/gs/consuming-rest/>

Consuming RESTful API in Python:

<http://stackoverflow.com/questions/17301938/making-a-request-to-a-restful-api-using-python>

Sending HTTP/POST get using plain TCP sockets:

<http://stackoverflow.com/questions/22077802/simple-c-example-of-doing-an-http-post-and-consuming-the-response>

An example of using PuTTY to make a basic “raw socket” request to the API:



```
GET /api/system/ HTTP/1.1
Host: LOCALHOST:9901
Authorization: Basic R0VXLVMxMjMONTowMDAwMDA=

HTTP/1.1 200 OK
Transfer-Encoding: chunked
Content-Type: text/json
Server: Microsoft-HTTPAPI/2.0
Date: Thu, 17 Nov 2016 14:20:42 GMT

60
{"SystemSerial":"S12345","SystemDescription":"E2C-45-6","CurrentDateTime":"17/11/2016 14:20:42"}
0
```

NOTES:

NOTES: